CDS Berkeley DevOps Assignment

Author	 Irfath Ramiz
Tags	
Contact Email	irfathr@gmail.com

Overview

This assignment demonstrates a production-ready, multi tier web application deployed on a Kubernetes environment (EKS - Amazon managed Kubernetes cluster) using Infrastructure as Code (IaC). The application consists of a Python Flask frontend that interacts with an in-memory Redis data to track and display a persistent visitor counter.

Used Tools and Tech Stack

Used AWS Services

- Amazone Elastic Kubernetes Service - EKS
- Amazon Elastic Container Registry (ECR).
- Network Loadbalancer
- VPC
- Amazone EC2
- IAM

VSCode

Application Layer

- Python
- Redis

Version Control

- Github / Glt

Infrastructure and provisioning

- Terraform 1.14.8
- AWS CLI V2

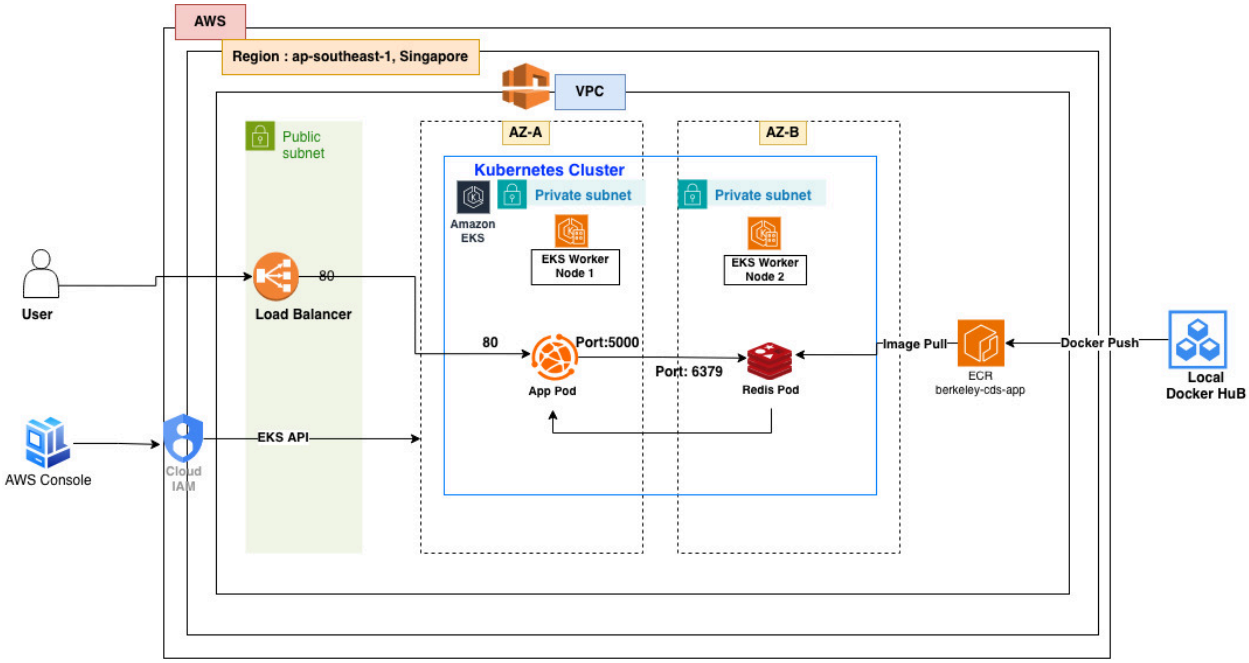
Containerization & Registry

- Docker
- ECR - Amazon Elastic Container Registry

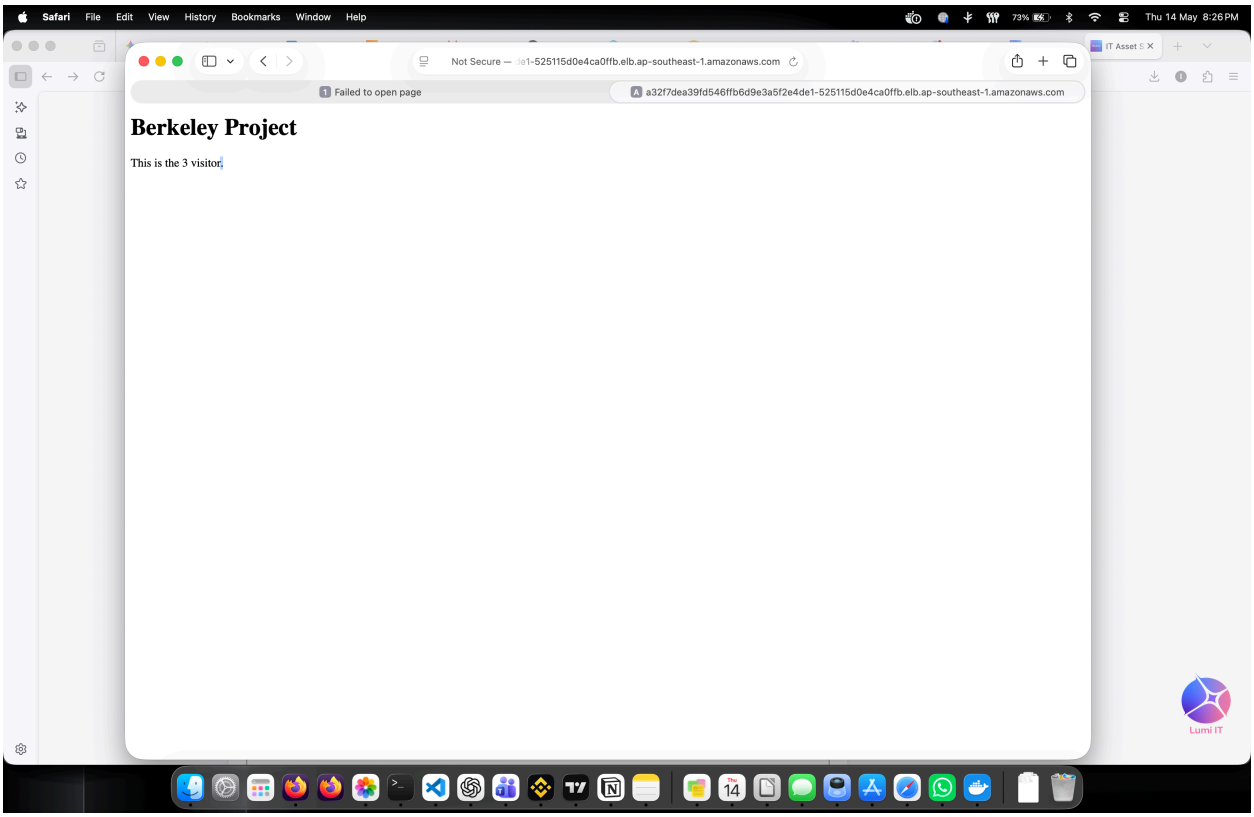
Kubernetes V 1.30.5

- EKS

Architecture



URL : http://a32f7dea39fd546ffb6d9e3a5f2e4de1-525115d0e4ca0ffb.elb.ap-southeast-1.amazonaws.com



Configuration & Deployment Steps

Folder Structure

```
.
├── Docs
│   └── berkeley-CDS-assignment-Architecture.drawio
├── app
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
├── docker-compose.yml
├── k8s
│   ├── deployment.yaml
│   ├── redis.yaml
│   └── service.yaml
└── terraform
  ├── berk1.tfplan
  ├── eks.tf
```

```
├── main.tf
├── provider.tf
├── terraform.tfstate
├── terraform.tfstate.backup
├── variables.tf
└── vpc.tf
```

Infrastructure Provitioning

use below commands to provision the Architecture on the AWS account.

```
cd terraaform

terraform init
terraform plan
terraform apply
```

Kubernetes Infrastructure

Multi-Environment Promotion (UAT/Prod)

This deployment has been deployed to “dev” namespace.

For the multi Environmental promotion i have used kubernetes namespaces to reduce the cloud cost since i have to be cost efficient as a free account.

UAT Environment

Bash

```
kubectl create ns uat
kubectl apply -f k8s/ -n uat
```

Prod Environment

Bash

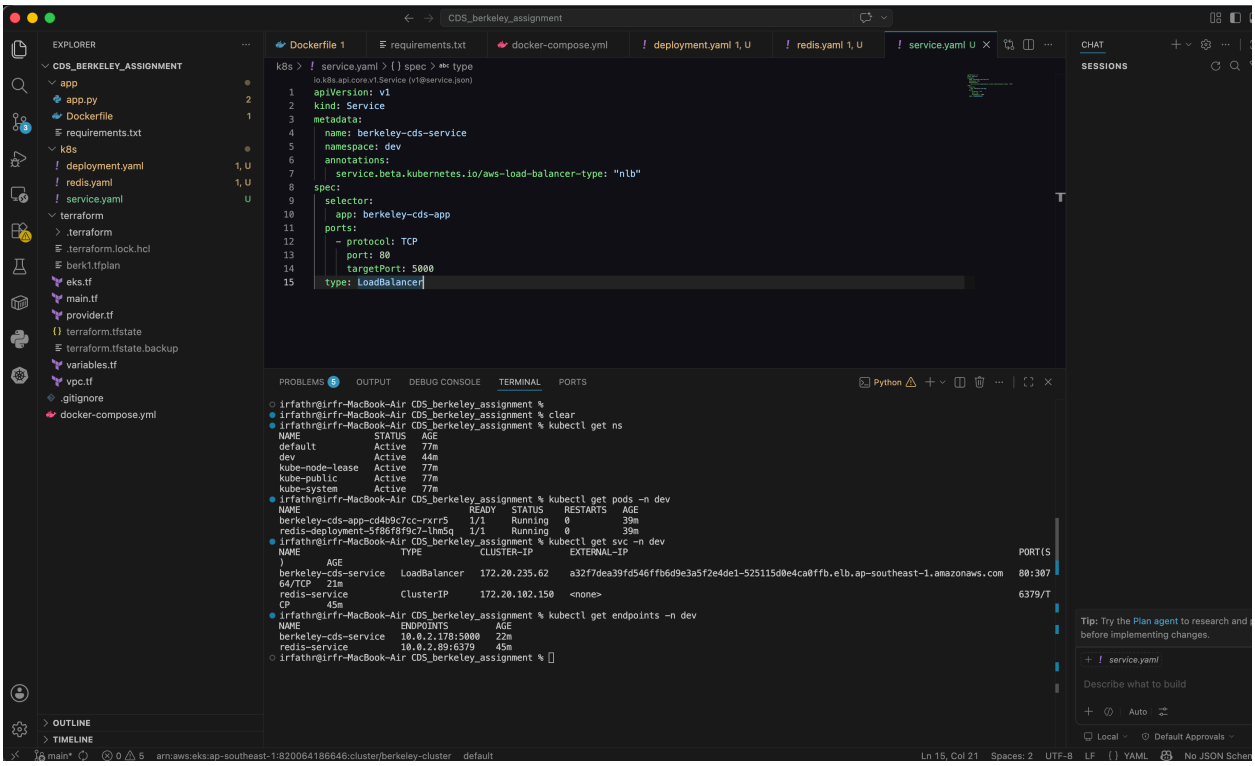
```
kubectl create ns prod
kubectl apply -f k8s/ -n prod
```

Error Fixing commands used

Here are the actual troubleshooting commands from my terminal history when I was verifying namespaces, logs, and diagnosing deployment issues:

Plaintext

```
206  kubectl get pods -n assessment
207  kubectl logs gateway-6f9f78d9d5-cgzj2
208  kubectl logs gateway-6f9f78d9d5-cgzj2 -n gateway-6f9f78d
9d5-cgzj2
209  kubectl logs gateway-6f9f78d9d5-cgzj2 -n assessment
210  kubectl rollout restart deployment gateway -n assessment
211  kubectl get pods -n assesment
355  kubectl describe pod gateway-6564d58766-x8vgc -n assessm
ent
```



Error Fixing commands used.

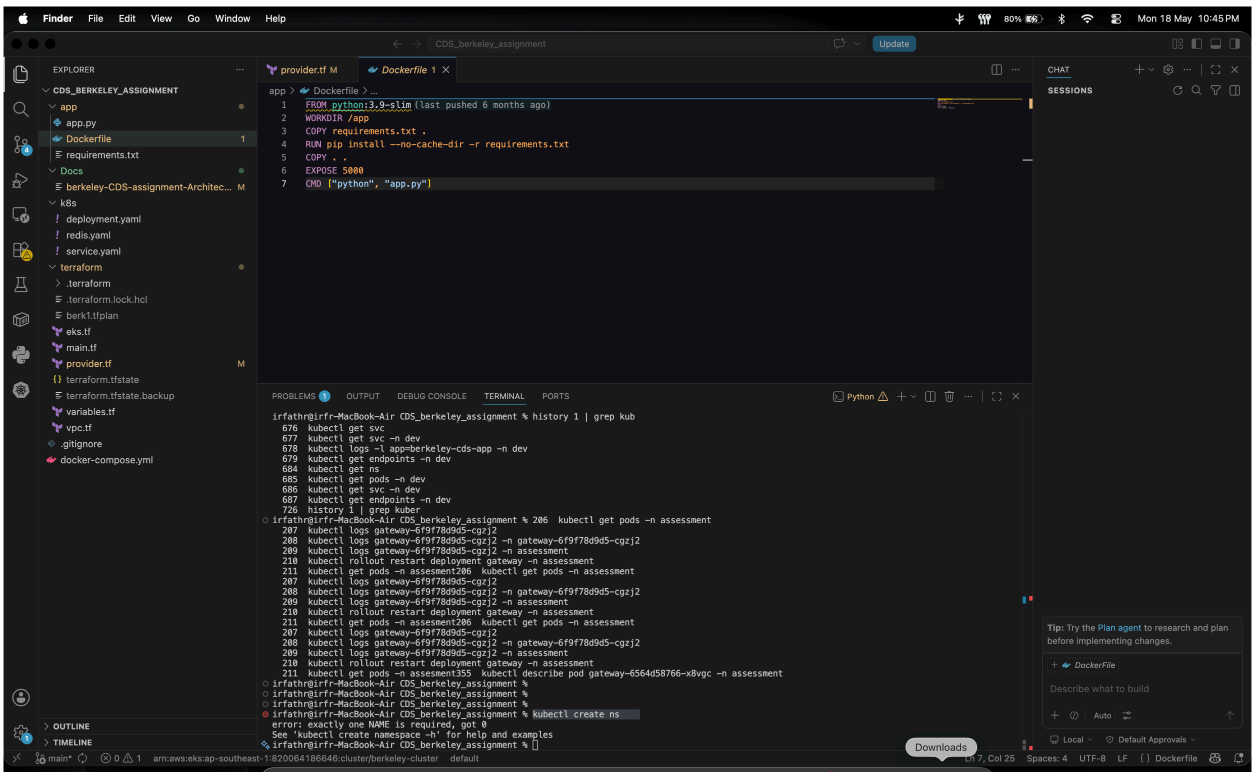
```
206  kubectl get pods -n assessment
207  kubectl logs gateway-6f9f78d9d5-cgzj2
208  kubectl logs gateway-6f9f78d9d5-cgzj2 -n gateway-6f9f78d
9d5-cgzj2
209  kubectl logs gateway-6f9f78d9d5-cgzj2 -n assessment
210  kubectl rollout restart deployment gateway -n assessment
211  kubectl get pods -n assesment

355  kubectl describe pod gateway-6564d58766-x8vgc -n assessm
ent
```

Best Practices I have Used.

Docker file Optimisations

- non Root User
- Minimal base image used



Future Improvements

Access Security Keys From hasicorp vault.

Terraform TF state file managements

Can store the state file in s3 bucket with versioning and store hcl lock file in a aws dynomodb for multi user executions. and best seruity managemt.

Terraform TF State File Management

tore the encrypted `terraform.tfstate` file in a private S3 bucket with Versioning enabled for safety and distribution and HCL LOCK file in a DynamoDB for multi user ecxecution.

Application Scalability & High Availability

In kuberneteys lare We can use HPA and vertical scaling options for availability.

Multi-Region Cloud Distribution for High Availability (HA)

Distribute the infrastructure globally across multiple AWS regions using an active-active availability zones.

Using Deployment tool - ArgoCD good for kubernetes infrastructures.

Container Storage - ECR

```
aws ecr create-repository --repository-name berkeley-cds—app
--region ap-southeast-1
```

Docker build

```
docker build --platform linux/amd64 -t cds_berkeley_assignmen
t-web:latest .
```

```
docker tag cds_berkeley_assignment-web:latest 820064186646.dk
r.ecr.ap-southeast-1.amazonaws.com/berkeley-cds-app:latest

docker push 820064186646.dkr.ecr.ap-southeast-1.amazonaws.co
m/berkeley-cds-app:latest
```

Alerts And System Monitoring

We are able to use Promethus amd Grafana dashboard systems to monitor the system avalibility error rates, and system performance.

In the AWS Console Cloud-watch.